Weekly Project Schedule - Stock Predictor

Week No.	Work Completed	Evidence Submitted	Issues Identified	Actions Taken	Work Planned for Next Week	Guide Remarks
1	Conducted literature review on stock prediction using LSTM, Transformer, and sentiment-based methods. Finalized project scope: AI-powered stock prediction with trading signals and risk metrics. Designed initial system flow (data -> features -> models -> API -> dashboard).	Research notes, architecture outline, README.md project plan, folder structure in backend/ and frontend/.	Difficulty in selecting a practical first implementation that can run on limited environment/dependencies.	Defined two execution modes: full AI pipeline (backend/main.py) and simplified demo mode (backend/main_simple.py) for reliable deployment.	Start data pipeline implementation and feature engineering from live market data.	
2	Implemented stock data ingestion using yfinance and utility modules for data preparation.	backend/utils/data_fetcher.py, backend/utils/feature_engineering.py, train_models.py, backend/config.py.	Data quality/availability variation across tickers and periods; preprocessing consistency required for training and	Standardized core feature set (OHLCV), scaling workflow, and sequence creation process used by both models.	Implement and train deep learning models (LSTM + Transformer), then persist artifacts.	

	Added feature engineering pipeline and sequence preparation for time-series learning (lookback window based). Added model training entry script for configurable ticker/period.		inference.			
3	Implemented LSTM model and Transformer model for price forecasting. Built ensemble predictor combining LSTM, Transformer, and sentiment impact with configurable weights. Integrated sentiment analysis module and prediction confidence	backend/models/lstm_model.py, backend/models/transformer_model.py, backend/models/ensemble_predictor.py, backend/utils/sentiment_analyzer.py.	Model import/runtime issues when heavy ML dependencies are unavailable in some environments.	Added defensive initialization in API routes with fallback handling; model loading logic supports pre-trained files and deferred training.	Integrate trading signal generation, risk analysis, and backtesting into API responses.	

	via model agreement.					
4	Built end-to-end API endpoints for prediction, backtest, and stock info. Added signal generation and risk management outputs (risk score, confidence, stop-loss, volatility). Developed frontend dashboard and static assets; wired backend to serve UI. Implemented simplified prediction/backtest endpoints for demo continuity.	backend/api/routes.py, backend/models/signal_generator.py, backend/models/risk_manager.py, backend/models/backtester.py, frontend/templates/index.html, frontend/static/js/main.js, frontend/static/css/style.css, backend/main_simple.py.	Training/deployment can be blocked by missing PyTorch or model files in target system.	Documented setup/training flow in README, added startup script (start.bat), and kept demo mode operational for immediate use.	Train models for selected tickers, evaluate backtest metrics, and improve model calibration/accuracy monitoring.	